

Assignment 3

Time-Series Data

Introduction

The purpose of this assignment was to apply recurrent neural networks (RNNs) to a time-series forecasting problem to see how different network architectures affect model performance. The time-series dataset used in this assignment was a weather forecasting dataset discussed in class.

There were multiple approaches used in this assignment including basic recurrent models as well as more advanced models such as stacked LSTMs and hybrid models. Since time-series data contains temporal dependencies, recurrent layers such as LSTM and GRU were used to see sequential patterns over time.

The main objective of this assignment was to improve forecasting performance by modifying the network architecture and determining which model produced the lowest validation mean absolute error (MAE).

Data and Test Setup

The dataset used in this assignment consists of weather time-series data. The input data was structured as sequences of past observations used to predict future value. For all tests the same training, validation, and test datasets were used. Model performance was evaluated using Mean Absolute Error (MAE). Validation MAE was used to select the best model and the best performing model was evaluated on the test dataset.

Several different models were tested

Test 1: Baseline: Simple Machine Learning

Test 2: LSTM (Single layer)

Test 3: Stacked LSTM with multiple layers and different units

Test 4: Conv1D + GRU hybrid model

Test 5: Conv1D + LSTM hybrid model

Methodology

Base Machine Learning Model

For the first test it was a simple machine learning model that was implemented using a flatten layer.

The model included

- One Flatten layer
- A dense output layer

The model was compiled using

- Loss function: Mean Squared Error (MSE)
- Optimizer: RMSprop
- Evaluation metric: MAE

LSTM Model

For the second test a simple LSTM model was implemented using a single recurrent layer.

The model included

- One LSTM Layer
- A dense output layer

The model was compiled using

- Loss function: Mean Squared Error (MSE)
- Optimizer: RMSprop
- Evaluation metric: MAE

Stacked LSTM Model

For the third test a stacked LSTM model was used to improve performance. Multiple LSTM Layers were added with decreasing unit sizes.

Example structure

- LSTM (64 units, return_sequences=True)
- LSTM (32 units)
- Dense output layer

Stacking allowed the network to learn more complex temporal patterns by passing the sequence information through multiple layers.

Conv1D + RNN Model

For additional tests convolutional layers were combined with recurrent layers.

The Conv1D layers were used:

- Extract short-term local patterns
- Reduce noise in the sequence

The RNN layers were then used either GRU or LSTM:

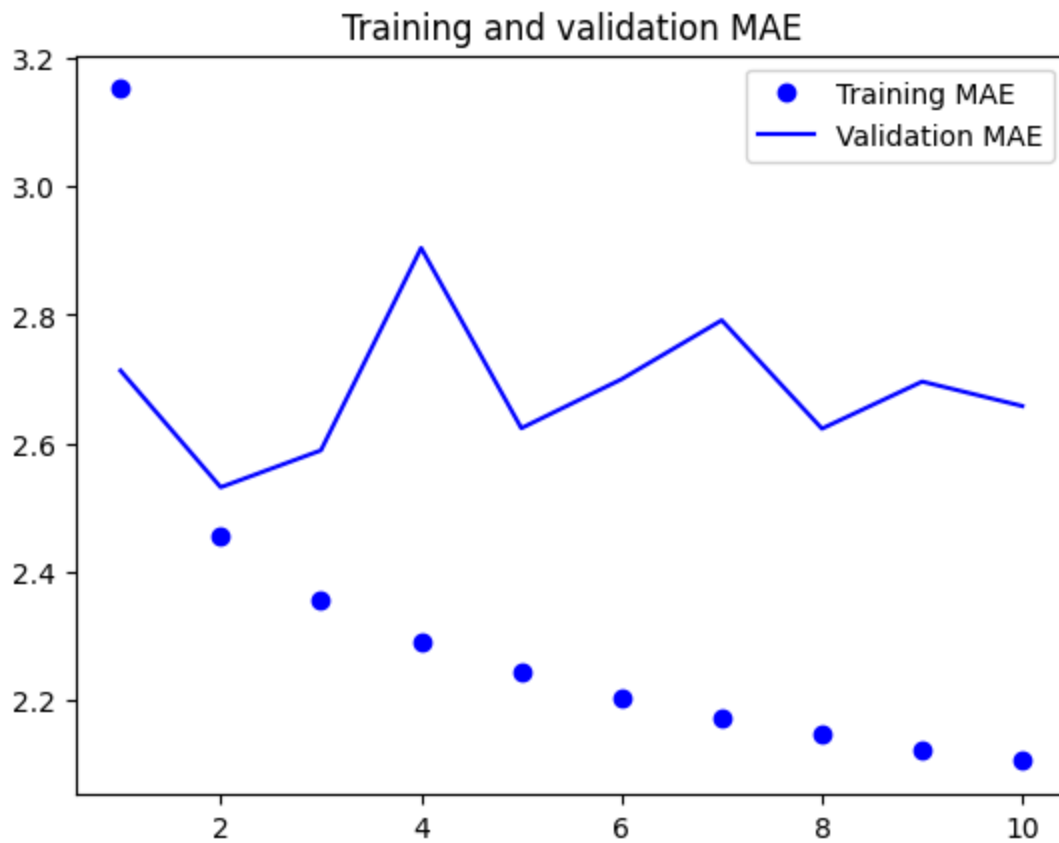
- Capture long-term temporal dependencies

The model structure included:

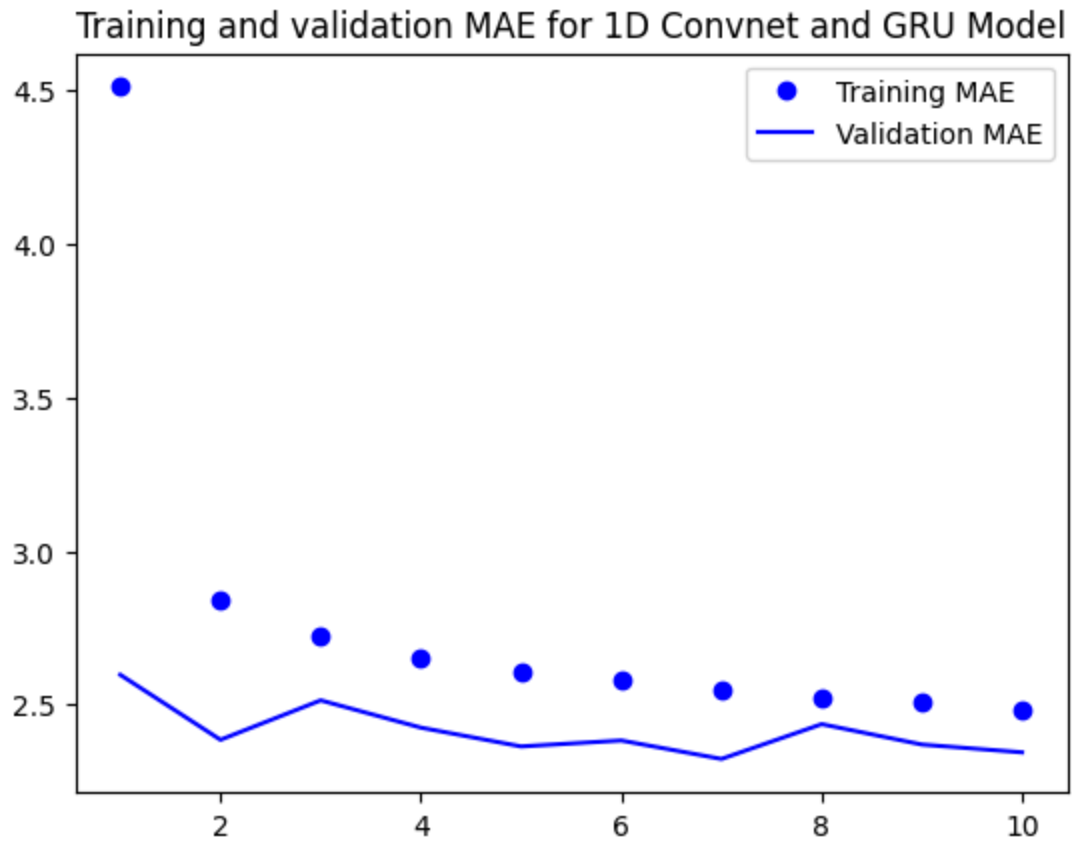
- Conv1D layer
- MaxPooling layer
- GRU or LSTM layer
- Dense output layer

Callbacks such as model checkpointing were used to save the best performing model during training.

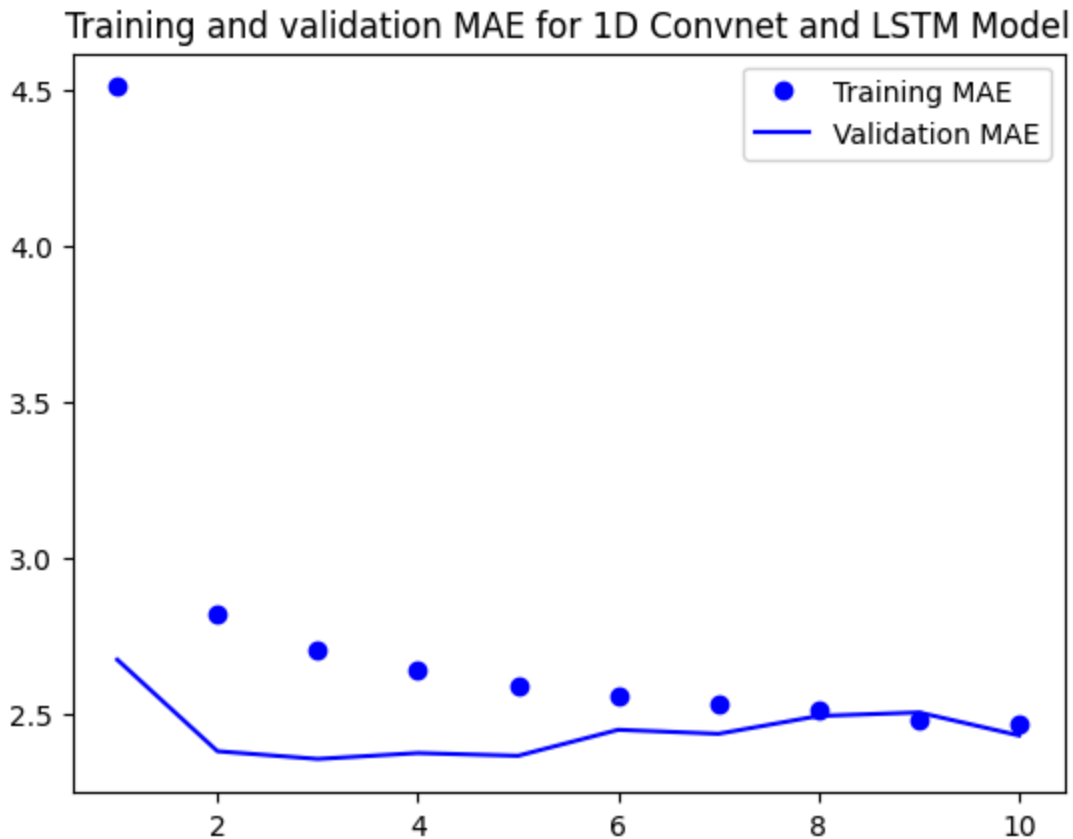
Test	Model Type	Key Change	Validation MAE	Test MAE
1	Baseline ML Model	Traditional ML	2.53	2.64
2	LSTM	Single layer (32 units)	2.41	2.53
3	Stacked LSTM	64 - 32 units	2.34	2.54
4	Conv1D + GRU	Hybrid model	2.32	2.49
5	Conv1D + LSTM	Hybrid model	2.35	2.50



In this figure you can see the baseline Machine Learning Model. The training MAE being around 2.10 after 10 epochs and the validation MAE being around 2.65 after 10 epochs.



In this figure you can see the hybrid Conv1D + GRU model. The training MAE after 10 epochs being around 2.48 and the validation MAE after 10 epochs being just below at 2.40 MAE.



In this figure you can see the hybrid Conv1D + LSTM model. The training MAE after 10 epochs is right around 2.50 MAE and the validation MAE after 10 epochs is around 2.43 MAE

Model Observations

Simple Machine Learning

The baseline machine learning model using a Flatten layer performed the worst among all models. This is likely because it does not capture temporal dependencies in the data, treating the sequence as independent features rather than a time-series.

LSTM

The model provided a reasonable starting point for LSTM but had limited ability to capture more complex temporal relationships due to its simple structure

Stacked LSTM

The stacked LSTM model improved performance by learning hierarchical temporal patterns. Increasing the number of layers and varying unit sizes helped reduce validation MAE compared to the baseline.

Conv1d + RNN Models

The hybrid models combining Conv1D and RNN layers showed improved performance in some cases. The convolutional layers helped extract meaningful local features. This helped improve the ability of the recurrent layers to model the sequence.

Discussion

The results demonstrate that the model architecture plays an important role in time-series forecasting performance.

The LSTM model performed well but was limited in capturing complex temporal dependencies. By stacking multiple LSTM layers the model was able to learn more detailed sequence patterns resulting in improved performance.

The Conv1D + RNN models introduce an additional improvement by combining feature extraction with sequence modeling. The convolutional layers helped identify short term trends while the recurrent layer captured longer term dependencies.

However increasing the model complexity does not always guarantee better performance. In some cases deeper models may lead to overfitting. This can negatively impact validation MAE.

Overall, the best performing model based on validation MAE was the Conv1D + GRU and then evaluated on the test dataset confirming the ability to generalize to unseen data.

Conclusion

In this assignment recurrent neural networks were applied to a weather time-series forecasting problem using multiple models.

The results showed that:

- Stacked LSTM models improved performance compared to a single layer LSTM
- Hybrid Conv1D + RNN models can further improve performance by combining feature extraction and sequence learning
- Model architecture significantly impacts forecasting accuracy

The best performing model being Conv1D + GRU achieved the lowest validation MAE and demonstrated strong performance on the test dataset.

Overall, this assignment highlights the importance of testing with different deep learning models when working with time-series data and shows that combining multiple techniques can lead to improved forecasting results.